

Algorithms

Algorithms

Definition 1: An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.

Example: Describe an algorithm using **pseudocode** for finding the maximum (largest) value in a finite sequence of integers.

ALGORITHM 1: Finding the Maximum Element in a Finite Sequence.

procedure *max* ($a_1, a_2, \dots, a_n$: integers)

max := a_1

for $i := 2$ **to** n

if $max < a_i$ **then** $max := a_i$

{*max* is the largest element}

Searching Algorithms

Searching problem: Locate an element x in a list of distinct elements $a_1, a_2, \dots, a_n$, or determine that it is not in the list. The solution to this search problem is the location of the term in the list that equals x (that is, i is the solution if $x = a_i$) and is 0 if x is not in the list.

THE LINEAR SEARCH

ALGORITHM 2: The Linear Search Algorithm.

procedure *linear search* (x : integer, $a_1, a_2, \dots, a_n$: distinct integers)

$i := 1$

while ($i \leq n$ and $x \neq a_i$)

$i := i + 1$

if $i \leq n$ **then** $location := i$

else $location := 0$

{ $location$ is the subscript of the term that equals x , or is 0 if x is not found}

THE BINARY SEARCH

The **binary search algorithm** can be used when the list has terms occurring in order of increasing value.

Example: Search for 19 in the list

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	Index
1	2	3	5	6	7	8	10	12	13	15	16	18	19	20	22	Data

$$m = \lfloor (1 + 16)/2 \rfloor = 8 \text{ (middle point)}$$

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	2	3	5	6	7	8	10	12	13	15	16	18	19	20	22

Since $19 > 10$, solution is in the second portion

9	10	11	12	13	14	15	16
12	13	15	16	18	19	20	22

$$m = \lfloor (9 + 16)/2 \rfloor = 12$$

9	10	11	12	13	14	15	16
12	13	15	16	18	19	20	22

Since $19 > 16$, solution is in the second portion

Example (contd.): Search for 19 in the list

13	14	15	16
18	19	20	22

$$m = \lfloor (13 + 16)/2 \rfloor = 14$$

13	14	15	16
18	19	20	22

Since 19 not > 19, solution is in the first portion

13	14
18	19

$$m = \lfloor (13 + 14)/2 \rfloor = 13$$

13	14
18	19

Since 19 > 18, solution is in the second portion

14
19

19 is the 14th term of the list. Return value is 14.

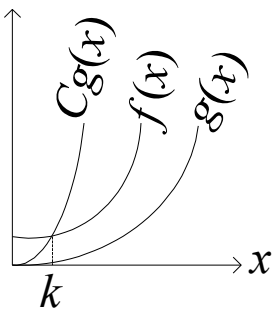
ALGORITHM 3: The Binary Search Algorithm.

```
procedure binary search ( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
 $i := 1$       { $i$  is left endpoint of search interval}
 $j := n$       { $j$  is right endpoint of search interval}
while  $i < j$ 
begin
     $m := \lfloor (i + j) / 2 \rfloor$       {  $m$  is middle point of search interval}
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
end
if  $x = a_i$  then  $location := i$ 
else  $location := 0$ 
{ $location$  is the subscript of the term that equals  $x$ , or is 0 if  $x$  is not found}
```

The Growth of Functions

Big- O Notation

Definition 1: Let f and g be functions from the set of non-negative integers or the set of non-negative real numbers to the set of non-negative real numbers. We say that $f(x) = O(g(x))$ if there are constants C and k such that $f(x) \leq C g(x)$, whenever $x \geq k$.



The constants C and k in the definition of big- O notation are called **witness** to the relationship $f(x) = O(g(x))$.

To establish that $f(x) = O(g(x))$ we need only one pair of witnesses to this relationship.

Example: Show that $f(x) = x^2 + 2x + 1 = O(x^2)$.

Solution:

$$\begin{aligned} f(x) = x^2 + 2x + 1 &\leq x^2 + 2x^2 + x^2 & [x \leq x^2, 1 \leq x^2 \text{ for } x \geq 1] \\ &= 4x^2 \end{aligned}$$

When $k = 1$, $C = 4$, and $g(x) = x^2$, $f(x) = x^2 + 2x + 1 \leq 4x^2$.

$$\therefore f(x) = x^2 + 2x + 1 = O(x^2).$$

When big- O notation is used, the function g in the relationship $f(x) = O(g(x))$ is chosen to be **as small as possible** (sometimes from a set of reference functions)

Some Important Big- O Results

Theorem 1: Let $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, where $a_0, a_1, \dots, a_{n-1}, a_n$ are real numbers with $a_n \neq 0$. Then $f(x) = O(x^n)$.

Proof: Omitted

Example: How can big- O notation be used to estimate the sum of the first n positive integers?

Solution:

$$1 + 2 + \dots + n \leq n + n + \dots + n = n^2$$

For $k = 1$ and $C = 1$

$$1 + 2 + \dots + n = O(n^2)$$

Example: Give big- O estimate for the factorial function and the logarithm of the factorial function, where the factorial function $f(n) = n!$ is defined by $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$, whenever n is a positive integer, and $1! = 1$.

Solution:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n \leq n \cdot n \cdot n \cdot \dots \cdot n = n^n$$

For $k = 1$ and $C = 1$

$$n! = O(n^n)$$

$$n! \leq n^n$$

$$\text{or, } \log n! \leq \log n^n = n \log n$$

For $k = 1$ and $C = 1$

$$\log n! = O(n \log n)$$

Big- O notation is used to estimate the number of operations needed to solve a problem using a specified procedure or algorithm. The functions used in these estimates often include the following:

$1, \log n, n, n \log n, n^2, 2^n, n!$

Each function in the list is smaller than the succeeding function.

Example:

Let $f(x) = c$, where c is a positive constant

$f(x) = c \leq c \cdot 1$, where $k = 0$, $C = c$, and $g(x) = 1$

$\therefore f(x) = c = O(1)$

Big- O of any constant is $O(1)$.

The Growth of Combination of Functions

Theorem 2: Suppose that $f_1(x) = O(g_1(x))$ and $f_2(x) = O(g_2(x))$. Then $(f_1 + f_2)(x) = O(\max(g_1(x), g_2(x)))$

Proof: Omitted

Theorem 3: Suppose that $f_1(x) = O(g_1(x))$ and $f_2(x) = O(g_2(x))$. Then $(f_1 f_2)(x) = O(g_1(x)g_2(x))$.

Proof: Omitted

Example: Give a big- O estimate for $f(n) = 3n\log(n!) + (n^2 + 3)\log n$, where n is a positive integer.

Solution:

$$3n = O(n)$$

$$\log(n!) = O(n\log n)$$

$$\therefore 3n\log(n!) = O(n(n\log n)) = O(n^2\log n)$$

$$(n^2 + 3) = O(n^2)$$

$$\log n = O(\log n)$$

$$\therefore (n^2 + 3)\log n = O(n^2(\log n)) = O(n^2\log n)$$

$$\begin{aligned}\therefore f(n) &= 3n\log(n!) + (n^2 + 3)\log n \\ &= O(\max(n^2\log n, n^2\log n)) = O(n^2\log n)\end{aligned}$$

Example: Give big- O estimate for $f(x) = (x + 1)\log(x^2 + 1) + 3x^2$

Solution:

$$(x + 1) = O(x)$$

$$\begin{aligned}\log(x^2 + 1) &\leq \log(x^2 + x^2) \text{ [when } x \geq 1\text{]} \\ &= \log(2x^2) \\ &= \log 2 + \log x^2 \\ &= \log 2 + 2\log x \\ &\leq \log x + 2\log x \text{ [when } x \geq 2\text{]} \\ &= 3\log x\end{aligned}$$

$$\therefore \log(x^2 + 1) = O(\log x) \text{ [} k = 2 \text{ and } C = 3\text{]}$$

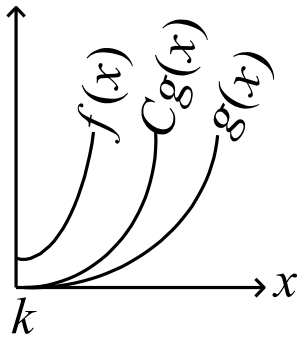
$$\therefore (x + 1)\log(x^2 + 1) = O(x\log x)$$

$$3x^2 = O(x^2)$$

$$\therefore f(x) = (x + 1)\log(x^2 + 1) + 3x^2 = O(\max(x\log x, x^2)) = O(x^2)$$

Big-Omega and Big-Theta Notation

Definition 2: Let f and g be functions from the set of non-negative integers or the set of non-negative real numbers to the set of non-negative real numbers. We say that $f(x) = \Omega(g(x))$ if there are constants C and k such that $f(x) \geq C g(x)$, where $x \geq k$.



Example:

$$f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3 \text{ [when } x \geq 0]$$

$$\therefore f(x) = 8x^3 + 5x^2 + 7 = \Omega(x^3) \text{ [} k = 0 \text{ and } C = 8]$$

Definition 3: Let f and g be functions from the set of non-negative integers or the set of non-negative real numbers to the set of non-negative real numbers. We say that $f(x) = \theta(g(x))$ if $f(x) = O(g(x))$ and $f(x) = \Omega(g(x))$.

Example: Show that $3x^2 + 8x \log x = \theta(x^2)$.

Solution:

$$3x^2 + 8x \log x \leq 3x^2 + 8xx = 3x^2 + 8x^2 = 11x^2 \text{ [when } x \geq 1]$$

$$\therefore 3x^2 + 8x \log x = O(x^2) \text{ [} k = 1 \text{ and } C = 11]$$

$$3x^2 + 8x \log x \geq 3x^2 \text{ [when } x \geq 1]$$

$$\therefore 3x^2 + 8x \log x = \Omega(x^2) \text{ [} k = 1 \text{ and } C = 3]$$

$$\therefore 3x^2 + 8x \log x = \theta(x^2)$$

Theorem 4: Let $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, where $a_0, a_1, \dots, a_{n-1}, a_n$ are real numbers with $a_n \neq 0$. Then $f(x) = \theta(x^n)$.

Example:

$$3x^8 + 10x^7 + 221x^2 + 1444 = \theta(x^8)$$

$$x^{19} - 18x^4 - 10112 = \theta(x^{19})$$

Complexity of Algorithms

Time Complexity

The **time complexity** of an algorithm is an estimate of the time required by the algorithm to solve a problem of a particular size.

The time complexity of an algorithm can be expressed in terms of the number of operations used by the algorithm when the input has a particular size.

WORST-CASE COMPLEXITY

By the **worst-case** complexity of an algorithm, we mean the largest number of operations needed to solve the given problem using this algorithm on input of specified size.

AVERAGE-CASE COMPLEXITY

In **average-case** complexity, the average number of operations used to solve the problem over all inputs of a given size is found.

Example: Describe the time complexity of Algorithm 1 for finding the maximum element in a set.

Solution:

ALGORITHM 1: Finding the Maximum Element in a Finite Sequence.

procedure *max* ($a_1, a_2, \dots, a_n$: integers)

max := a_1

For $i := 2$ **to** n

if *max* < a_i **then** *max* := a_i

{*max* is the largest element}

The **for** loop will be executed $(n - 1)$ times.

For each **for** loop, two comparisons will be done.

One more comparison will be needed to exit the **for** loop.

Therefore, exactly $2(n - 1) + 1 = 2n - 1$ comparisons are used.

Hence, the average-case time complexity of the algorithm is $2n - 1 = \theta(n)$.

Example: Describe the worst-case time complexity of the linear search algorithm.

Solution:

ALGORITHM 2: The Linear Search Algorithm.

```
procedure linear search (x: integer,  $a_1, a_2, \dots, a_n$ : distinct integers)
i := 1
while ( $i \leq n$  and  $x \neq a_i$ )
    i := i + 1
if  $i \leq n$  then location := i
else location := 0
{location is the subscript of the term that equals x, or is 0 if x is not found}
```

At each step of the **while** loop, two comparisons are performed.
One more comparison is made outside the loop.

If $x = a_i$, then $2i + 1$ comparisons are used.

When the element is not in the list, $2n + 1 + 1 = 2n + 2$ comparisons are required.

Hence, the worst-case time complexity of linear search algorithm is $2n + 2 = O(n)$.

Example: Describe the time complexity of the binary search algorithm.

Solution:

ALGORITHM 3: The Binary Search Algorithm.

```
procedure binary search ( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
 $i := 1$       {  $i$  is left endpoint of search interval }
 $j := n$       {  $j$  is right endpoint of search interval }
while  $i < j$ 
begin
     $m := \lfloor (i + j) / 2 \rfloor$       {  $m$  is middle point of search interval }
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
end
if  $x = a_i$  then  $location := i$ 
else  $location := 0$ 
{  $location$  is the subscript of the term that equals  $x$ , or is 0 if  $x$  is not found }
```

Example: Describe the time complexity of the binary search algorithm. (Contd.)

Assume $n = 2^k$, where k is a nonnegative integer

$$\therefore k = \log n$$

<u>Search Interval Length</u>	<u>Comparisons</u>
2^k	2
2^{k-1}	2
2^{k-2}	2
...	...
2^2	2
2^1	2
$2^0 = 1$	1

One more comparison is used to determine if this item is x .

Thus, binary search algorithm requires

$$2k + 1 + 1 = 2k + 2 = 2\log n + 2 \text{ comparisons.}$$

Hence, the average-case time complexity of binary search is $2\log n + 2 = \theta(\log n)$.

Example: Describe the average-case complexity of the linear search algorithm, assuming that the element x is in the list.

Solution:

ALGORITHM 2: The Linear Search Algorithm.

procedure *linear search* (x : integer, $a_1, a_2, \dots, a_n$: distinct integers)

$i := 1$

while ($i \leq n$ and $x \neq a_i$)

$i := i + 1$

If $i \leq n$ **then** $location := i$

else $location := 0$

{ $location$ is the subscript of the term that equals x , or is 0 if x is not found}

<u>Location of x</u>	<u>Comparison needed</u>
1	$2 \times 1 + 1 = 3$
2	$2 \times 2 + 1 = 5$
...	...
n	$2n + 1$

Example: Describe the average-case complexity of the linear search algorithm, assuming that the element x is in the list. (contd.)

Thus, the average number of comparisons used by the linear search algorithm (when x is known to be in the list) is

$$\frac{(2 \times 1 + 1) + (2 \times 2 + 1) + \square + (2n + 1)}{n} = \frac{2(1 + 2 + \square + n) + (1 + 1 + \square + 1)}{n} = \frac{2 \left[\frac{n(n+1)}{2} \right] + n}{n} = (n + 1) + 1 = n + 2$$

Hence, the average-case time complexity of linear-search algorithm is $n + 2 = \theta(n)$.